AI based Release Risk Predictor

Live repo: https://github.com/nanantha17/AI-release-risk-predictor

The Problem

In large-scale software NPI programs, dozens of patches are integrated simultaneously across software teams and cross-programs. Each patch brings unknown risk:

- A single bad integration can cause production downtime
- Teams have no visibility into which patch introduced instability until it's too late
- CI/CD pipelines surface failures only after the fact — not before the decision to integrate
- Release managers are forced to make Go/NoGo decisions with incomplete, scattered signals

The result: last-minute integration failures, emergency hotfixes, delayed customer launches, and escalations that cost millions.

The Solution

An end-to-end AI system that scores the risk brought in by each software patch before integration — giving CI/CD pipelines and release managers an early warning signal to act on.

1. Per-patch risk scoring from 13 quality signals (test coverage, defect history, CI metrics, code complexity, release notes)
2. Explainable risk factors — not just a score, but *which signals* are driving risk and by how much which is quantifiable
3. Actionable recommendations— specific steps to reduce risk before the patch is integrated
4. Ensemble ML confidence— three independent models must agree before flagging CRITICAL

Real-world impact at ASML: 40% reduction in integration issues by identifying high-risk patches early and redirecting team effort before they reached CI/CD saving millions $

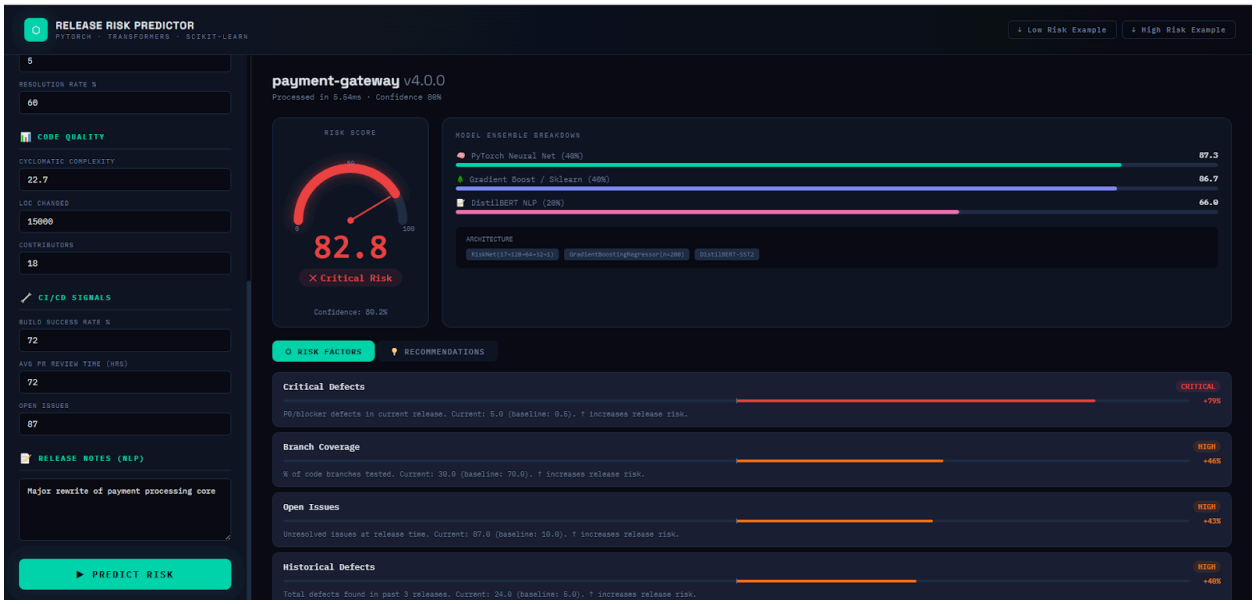
An ensemble ML system that predicts software release risk from 13 quality metrics, explains the top risk factors, and recommends actions — deployed as a REST API with an interactive React dashboard.

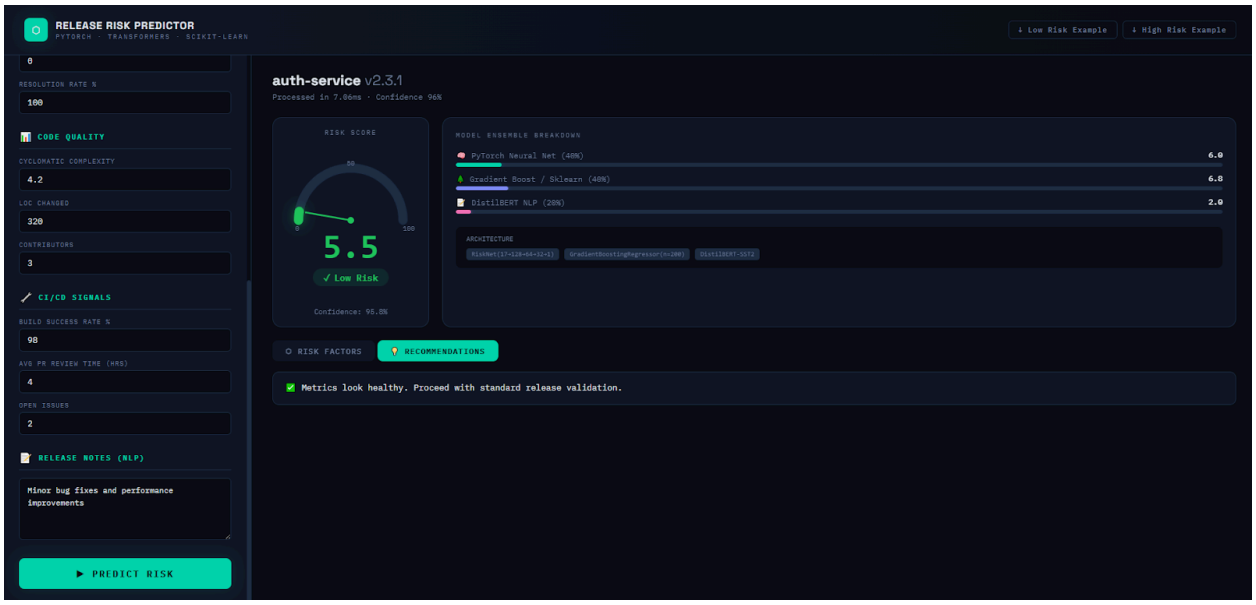
The end-to-end ML application predicts software release risk using an ensemble of:

1. PyTorch deep neural network (tabular metrics)
2. Scikit-learn Gradient Boosting Regressor

3. HuggingFace Transformers DistilBERT for release notes NLP

Dashboard





How It Works

If I feed it my patch/release metrics → I get back a risk score, confidence level, ranked risk factors, and actionable recommendations in under 1 second.

Example: Catastrophic release

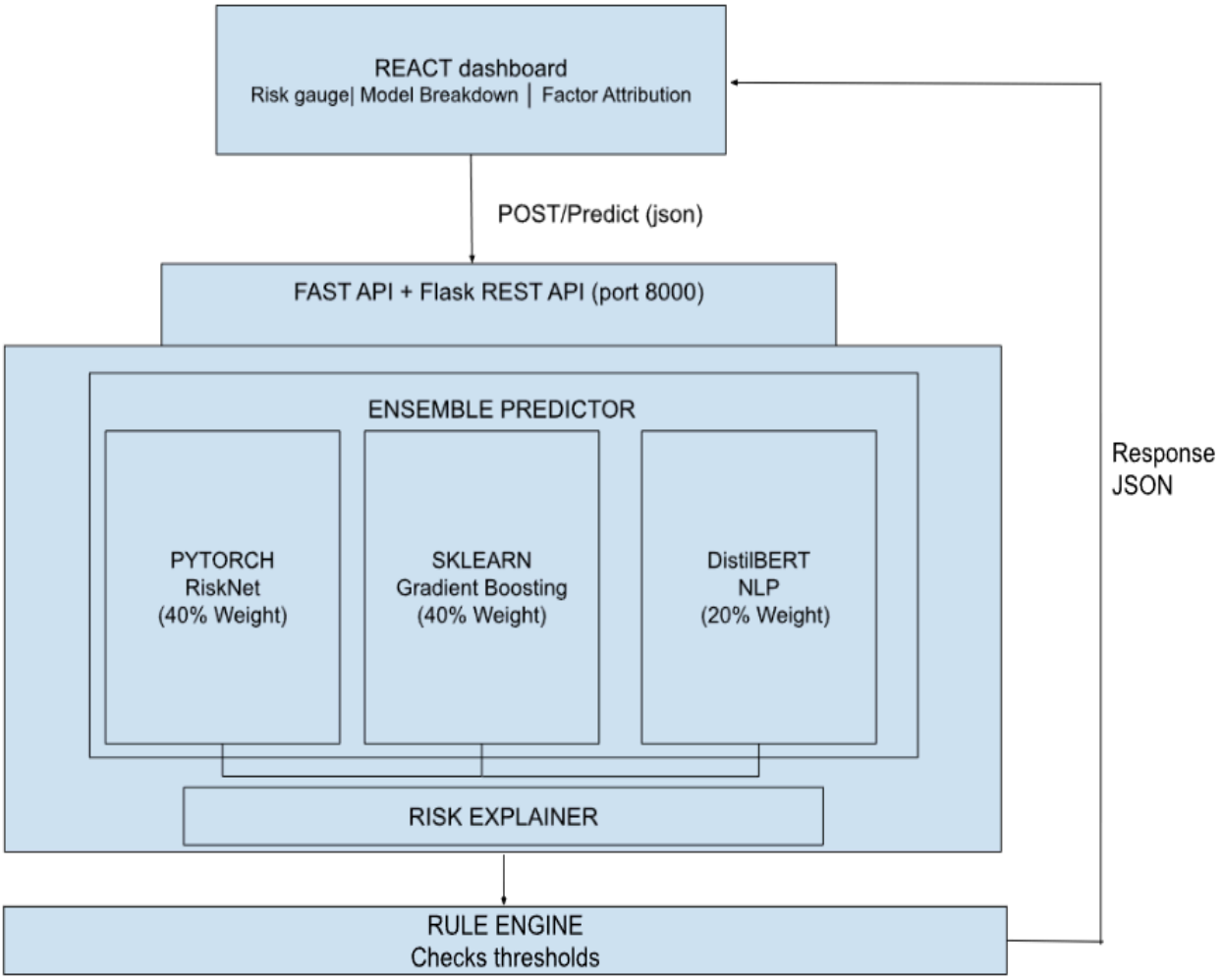
```json
{
  "risk_score": 88.4,
  "risk_level": "CRITICAL",
  "confidence": 0.97,
  "ml_breakdown": {
    "pytorch_neural_net": 87.9,
    "sklearn_gradient_boost": 87.6,
    "nlp_release_notes": 91.0
  }
}
```

Example: Healthy release

```json
{
  "risk_score": 5.3,
  "risk_level": "LOW",
  "confidence": 0.961,
  "ml_breakdown": {
    "pytorch_neural_net": 5.9,
    "sklearn_gradient_boost": 6.4,
    "nlp_release_notes": 2.0
  }
}
```

83-point spread between a catastrophic and a healthy release. All three models agree in both directions.

Architecture



ML Model Details

1. PyTorch RiskNet (40% weight)
- **Input:** 17 features (12 raw + 5 derived)
- **Architecture:** `Input(17) → BatchNorm → [256→128→64→32] → Sigmoid
  - Wider than original — better separation of extreme risk values
  - LayerNorm + GELU + Dropout at each block
- **Training:** 7,000 synthetic samples across 80 epochs
  - 6,000 base distribution samples
  - 500 deliberately extreme HIGH risk samples (forces model to learn the top end)
  - 500 deliberately extreme LOW risk samples (forces model to learn the bottom end)
- **Loss:** MSE + L1 + focal penalty on extreme samples
- **Scheduler:** CosineAnnealingLR (stepped per epoch)

- **Optimizer:** AdamW with weight decay, best-epoch checkpointing

📂 Project Structure

```
AI-release-risk-predictor/
├── backend/
│   ├── main.py              # FastAPI app, routes, Pydantic schemas
│   ├── requirements.txt
│   ├── saved_models/        # Auto-generated after first run (gitignored)
│   │   ├── risknet.pt       # PyTorch weights
│   │   └── sklearn_gbm.pkl  # Sklearn pipeline
│   └── models/
│       ├── __init__.py
│       ├── risk_model.py    # Ensemble: PyTorch + Sklearn + NLP
│       └── explainer.py     # SHAP-style feature attribution
└── frontend/
    ├── index.html
    ├── package.json
    ├── vite.config.js       # Dev server port 3000, proxy to :8000
    └── src/
        ├── main.jsx
        └── App.jsx          # React dashboard with gauge + breakdown
```